SEZIONE 2 · SOTTOSEZIONE 2.1

ARCHITETTURA, STACK ED EVOLUZIONE

2.1.1 INTRODUZIONE.

Founder's Dilemma è un business game di startup venturing per l'aula universitaria. Un solo sorgente TypeScript produce tre forme di distribuzione: app desktop Tauri, sito su GitHub Pages e file HTML unico. Il motore di simulazione è separato dall'interfaccia ed è una funzione pura di stato e decisioni (figura 2.1-1).

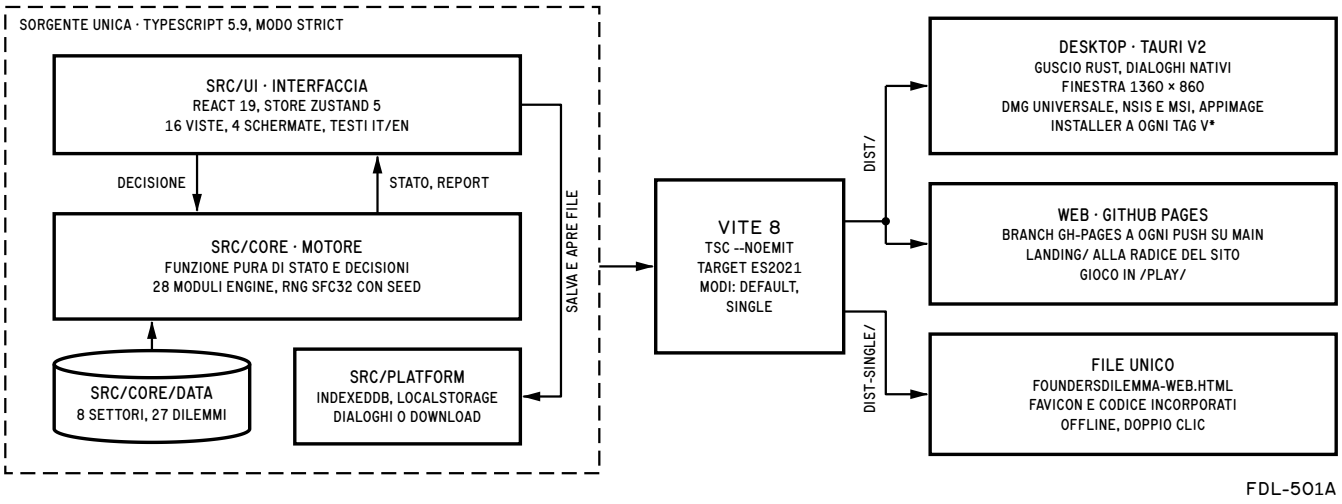


Figura 2.1-1. Schema a blocchi dello stack: un sorgente, tre forme di distribuzione

2.1.2 DESCRIZIONE FUNZIONALE.

Il motore (src/core) riceve lo stato della partita e le decisioni dei team e restituisce il nuovo stato con i report del round. L'interfaccia React legge lo stato da uno store Zustand. Il modulo src/platform salva le partite in IndexedDB e apre i file con i dialoghi nativi su desktop o con il download nel browser. Vite compila lo stesso codice in dist/ e, in modo single, in un file unico. Tre workflow GitHub Actions verificano, pubblicano e impacchettano (figura 2.1-2).

2.1.2.1 Separazione motore-interfaccia.

Nessun file di src/core importa da src/ui: lo stesso motore gira nei test Vitest in Node, nella simulazione Monte Carlo (npm run balance) e nell'app. Il guscio Rust conta 76 righe e serve solo a leggere e scrivere i file scelti dall'utente.

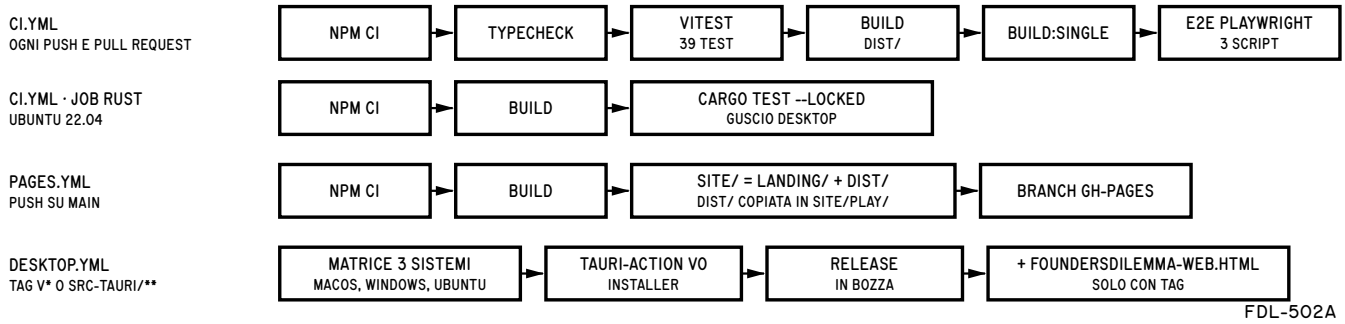


Figura 2.1-2. Flussi GitHub Actions di verifica, pubblicazione e rilascio

2.1.3 DATI DI PRESTAZIONE E DI PROGETTO.

Tabella 2.1-1. Componenti dello stack (versione 2.1.0)

Parametro	Valore
Linguaggio di motore e interfaccia	TypeScript 5.9, strict
Interfaccia e stato	React 19.3, Zustand 5
Build e test	Vite 8, Vitest 5, Playwright 1.64
Guscio desktop	Tauri 2, Rust 1.77.2 o succ.
Ambiente di sviluppo	Node.js 22, porta 1420
Finestra desktop (minima)	1360 × 860 (1024 × 680)
Righe TypeScript, motore / UI	9.922 / 12.815
Salvataggi locali	IndexedDB, store "kv"

ATTENZIONE

Il guscio desktop legge e scrive solo file .json, .csv, .txt e .html fino a 50 MB. Ogni altra estensione è rifiutata con "Unsupported file type".

NUOVA VERSIONE — MANUTENTORE

```
1 package.json - versione
2 Cargo.toml, tauri.conf - allineare
3 TAG v2.x.y - creare, push
  √ DESKTOP APPS - in corso
4 RELEASE BOZZA - verificare
  √ FILE WEB - allegato
5 RELEASE - pubblicare
```

FDL

2.1.4 EVOLUZIONE DEL PROGETTO.

Il progetto nasce il 12 gennaio 2026 come blueprint Swift e SwiftUI per macOS, con il nome VentureSim. Il 14 ottobre 2026 il motore è riscritto in TypeScript e il gioco diventa multipiattaforma. La storia del repository conta 38 commit distribuiti in cinque giornate di lavoro (figura 2.1-3).

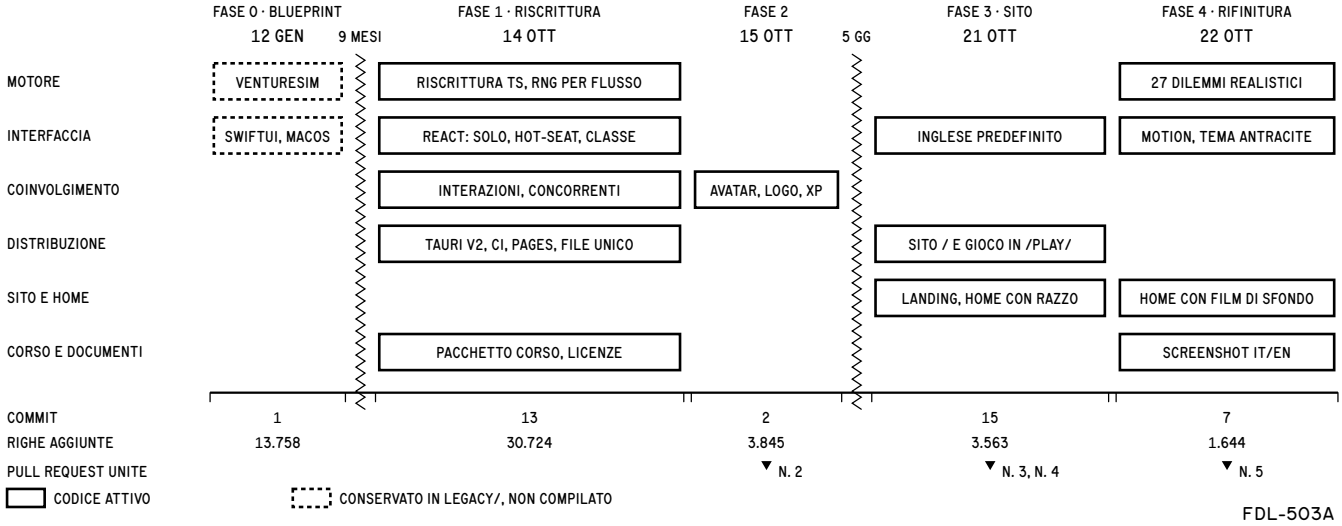


Figura 2.1-3. Cronologia dello sviluppo per area, giornate con commit

2.1.4.1 Fasi di sviluppo.

Il 14 ottobre arrivano in 13 commit il motore, l'interfaccia React con le tre modalità, l'app desktop, la CI, il pacchetto del corso e la pubblicazione su GitHub Pages. Il 15 ottobre si aggiunge lo strato di coinvolgimento: avatar, logo, XP, sfide di squadra e rete dei fondatori. Il 21 ottobre nascono la landing alla radice del sito e la home con il lancio del razzo; il 22 ottobre il passaggio su motion e UX, i 27 dilemmi realistici e il tema antracite.

2.1.4.2 Riscrittura del generatore casuale.

Il blueprint usava un unico LCG condiviso: le scelte di un team cambiavano le estrazioni di tutti gli altri. Il motore TypeScript deriva ogni flusso da seed, round, scopo e azienda (figura 2.1-4).

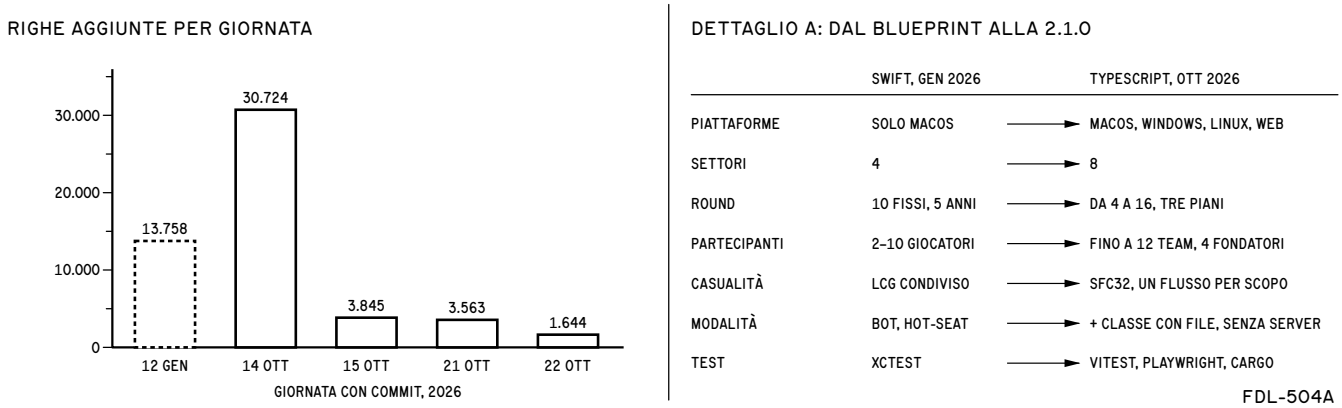


Figura 2.1-4. Volume di lavoro per giornata e confronto tra blueprint e versione attuale

2.1.5 DATI STORICI.

Tabella 2.1-2. Cronologia del repository

Parametro	Valore
Primo commit, blueprint Swift	12 gen 2026, 72 file
Sorgenti Swift del blueprint	45 file, 5.864 righe
Riscrittura TypeScript	14 ott 2026, 117 file
Commit totali, di cui merge	38, 6
Pull request unite	4 (n. 2–5)
Giornate con commit	5
Righe aggiunte / rimosse	53.534 / 2.415
Versione corrente	2.1.0

NOTA

La cartella legacy/ conserva il blueprint Swift e i video del mockup. La CI non la compila né la testa: è solo un riferimento storico.

AMBIENTE DI SVILUPPO	SVILUPPATORE

```
1 NODE.JS - 22 o successivo
2 npm ci - eseguire
3 npm run dev - eseguire
  √ localhost:1420 - risponde
4 npm test - eseguire
  √ VITEST - 39 test superati
5 npm run desktop:dev - con Rust
```

2.1.6 MOTORE DI SIMULAZIONE.

Il motore decide per round e simula per mese. Ogni round applica prima le azioni una tantum di ciascuna azienda, poi risolve insieme le operazioni che coinvolgono più team, infine simula da 3 a 6 mesi in cui tutti i team di un settore condividono lo stesso mercato (figura 2.1-5).

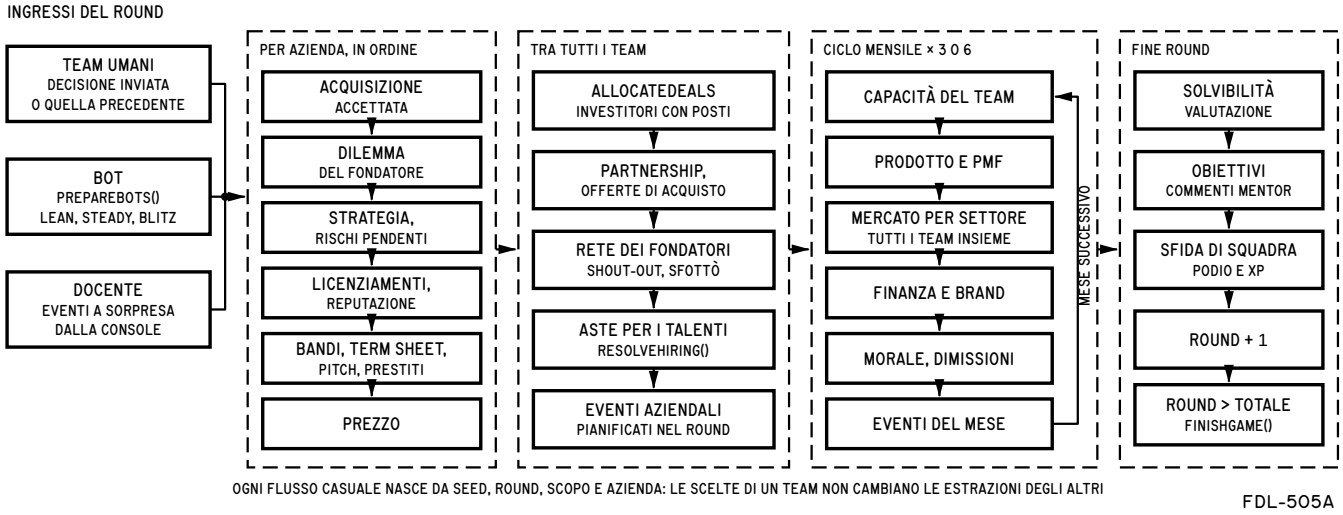


Figura 2.1-5. Ciclo del round in advanceRound(): azioni per azienda, azioni tra team, mesi simulati

2.1.6.1 Modalità Classe.

La modalità Classe funziona senza server. La copia del docente è la partita principale: il docente esporta il file di partita, ogni team gioca il round sul proprio portatile e restituisce un piccolo file di decisioni. Il docente importa i file, avanza il round e condivide il nuovo file. Solo e hot-seat girano invece su un unico computer (figura 2.1-6).

2.1.6.2 Anteprima onesta.

La proiezione mostrata durante la decisione usa una copia dello stato e un generatore a valore atteso: indica la tendenza senza rivelare gli esiti casuali del round.

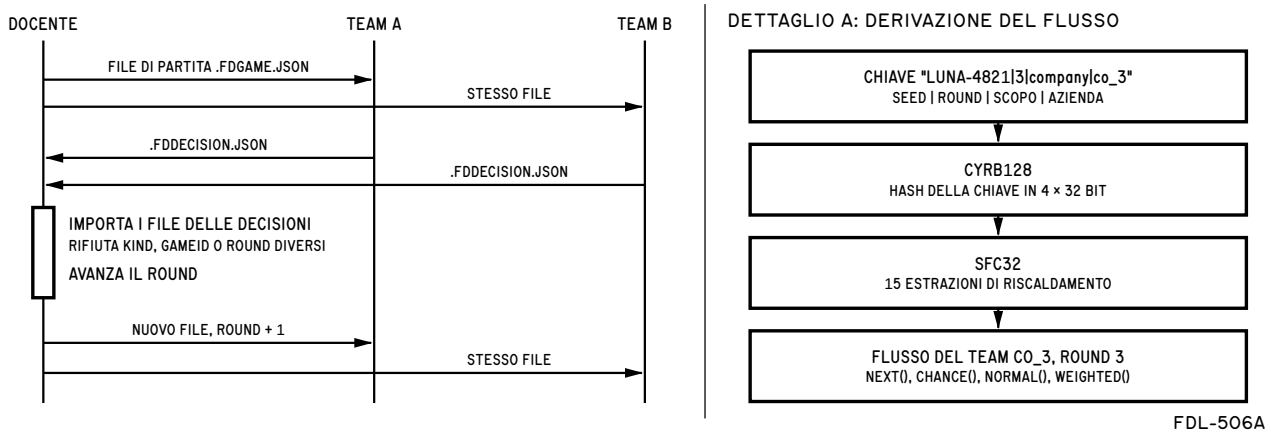


Figura 2.1-6. Modalità Classe senza server e derivazione dei flussi casuali

2.1.7 DATI DI PROGETTO DEL MOTORE.

Tabella 2.1-3. Parametri della partita

Parametro	Valore
Round per partita	da 4 a 16, predefinito 10
Mesi per round, piano startup	3 fino al round 4, poi 6
Durata della partita standard	48 mesi
Team per partita, di cui bot	fino a 12, fino a 8
Fondatori per team	da 1 a 4
Settori, dilemmi	8, 27
Scopi dei flussi casuali	11
Versione del formato di stato	3

ATTENZIONE

Un file di decisioni di un'altra partita o di un altro round è rifiutato. Importare tutti i file prima di avanzare: chi manca gioca con le decisioni precedenti.

ROUND IN CLASSE	DOCENTE
1 ESPORTA FILE PARTITA - premere	
2 FILE - condividere con i team	
3 IMPORTA I FILE - selezionare	
√ "Decisioni importate."	
4 AVANZA IL ROUND - premere	
5 rip. passi 1-4	

FDL